CEUE203-Object Oriented Programming 25AIML006

PRACTICAL-2

PART A:

Program – 1:

Smart thermostat (Thermostat.java, main in this file)

(a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount.

(c) Constructor(location): chain with this(location, 22).

(d) raise(): if temperature < MAX add 1, else print "Already at maximum (30)".

(e) lower(): if temperature > MIN subtract 1, else print "Already at minimum (16)".

(f) Add getTemperature() (no setter) and static getActiveCount().

(g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops
(printing the temperature each time), then print getActiveCount().
Expected output: temperature rises to 30 then "Already at maximum", falls to 16 then "Already at minimum",
count 2.

CODE:

```
public class Pract2_1 {
static class Thermostat {        private
String location;        private int
temp;        private static int Min =
16;        private static int Max = 30;
private static int activeCount=0;

    Thermostat(String location , int startTemp){
        this.location = location;
```

```
        if(startTemp >= Min && startTemp <= Max){
this.temp = startTemp;
```

```
        }else{
            this.temp = 22;
        }
        activeCount++;
    }

    Thermostat(String location){
this.location = location;
this.temp = 22;
        activeCount++;
    }

    public void raise() {
if (temp < Max) {
temp++;
        } else {
            System.out.println("Already at maximum : " + Max);
        }
    }

    public void lower() {
if (temp > Min) {
temp--;
        } else {
            System.out.println("Already at minimum  : "+ Min);
        }
    }

    public void getTemp(){
System.out.println(temp);
    }

    public void getCount(){
        System.out.println(activeCount);
    }
  }

  public static void main(String args[]){

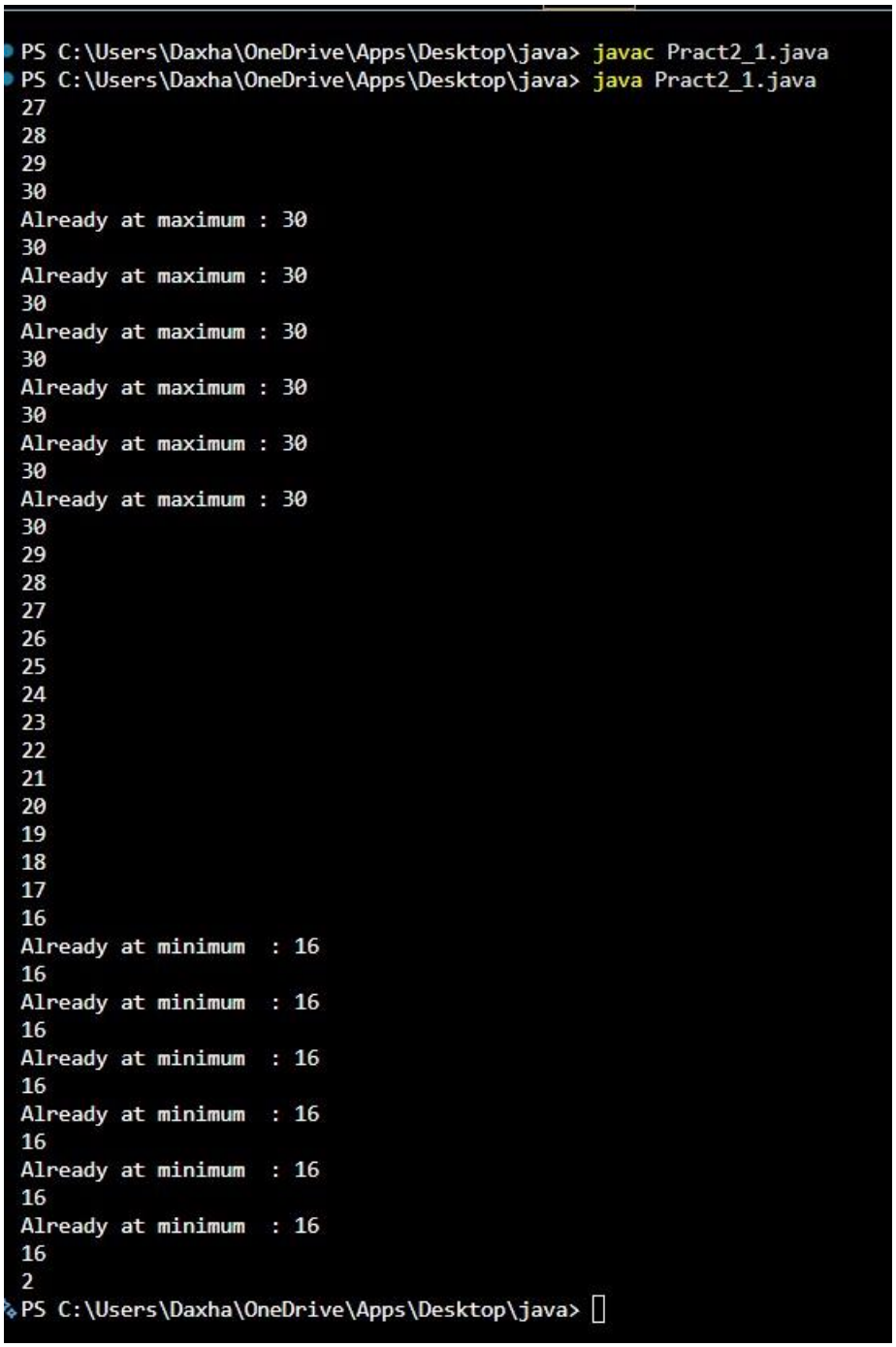
    Thermostat t1 = new Thermostat("Charusat",26);
    Thermostat t2 = new Thermostat("Charusat");
```

```
        for(int i = 1 ; i <= 10 ; i++){
t1.raise();
```

```
        t1.getTemp();
    }
    for(int i = 1 ; i <= 20 ; i++){
        t1.lower();
        t1.getTemp();
    }

    t1.getCount();
} }
```

OUTPUT:


PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> javac Pract2_1.java
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> java Pract2_1.java
27
28
29
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
Already at maximum : 30
30
29
28
27
26
25
24
23
22
21
20
19
18
17
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
Already at minimum : 16
16
2
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java>

Program – 2:

Cinema show (CinemaShow.java, main in this file)

(a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0.

(b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100).

(c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged.

(d) cancel(int n): increase seatsAvailable by n but never above capacity.

(e) Add getSeatsAvailable() and static getTotalBooked().

(f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked().

Expected output: booking more than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

Code :

```
public class Pract2_2 {     public static
class CinemaShow{         private
String title;         private int
seatAvailable;         private static int
totalBooked = 0;
        private final int capacity = 200;

        CinemaShow(String title , int capacity){
            this.title = title;
            this.seatAvailable = capacity;
        }
        CinemaShow(String title){
this(title,100);
        }

        public boolean book(int n){
if(n <= seatAvailable){
seatAvailable -= n;
totalBooked += n;                 return
true;
            }
            return false;
        }
```

```
        public void cancle(int seat){
            int a = seatAvailable + seat;
if(a <= capacity && a >= 0){
seatAvailable += seat;
                totalBooked -= seat;
            }

        }
    }

    public static void main(String[] args){
        CinemaShow c = new CinemaShow("Spyder Man : Brand New
Day",100);

        c.book(10);
        System.out.println(c.seatAvailable);
c.cancle(5);
        System.out.println(c.seatAvailable);

        System.out.println(c.totalBooked);

        }

}
```

Output :

```
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> javac Pract2_2.java
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> java Pract2_2.java
90
95
5
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java>
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java>
```

Program – 3:

(For Advanced Learners – home) Parking lot (ParkingLot.java)

(a) Create class ParkingLot with private int twoWheelers and fourWheelers, private final int twoCap and fourCap, and private static long revenue=0.

(b) park(String type): for "two", if twoWheelers < twoCap increment and add 20 to revenue; for "four", if fourWheelers < fourCap increment and add 40; otherwise print "Full" and reject.

(c) leave(String type): decrement the matching count but never below 0. (d) In main: simulate a sequence of park/leave events, printing each outcome, then print final occupancy and revenue.

Expected output: parking into a full section is refused; revenue = 20×(two parked)
+ 40×(four parked).

Code :

```
class Parkinglot {sss

   private int twoWheelers = 0;
   private int fourWheelers = 0;

   private final int twoCap;
   private final int fourCap;

   private static long revenue = 0;

   Parkinglot(int twoCap, int fourCap) {
this.twoCap = twoCap;
      this.fourCap = fourCap;
   }

   public void park(String type) {
if (type.equals("two")) {          if
(twoWheelers < twoCap) {
twoWheelers++;              revenue
+= 20;
            System.out.println("Two-wheeler parked");
         } else {
            System.out.println("Two-wheeler section Full");
         }
      } else if (type.equals("four")) {
```

```
            if (fourWheelers < fourCap)
{                   fourWheelers++;
revenue += 40;
                System.out.println("Four wheeler parked");
} else {
                System.out.println("Four wheeler section Full");
            }
        }
    }

    public void leave(String type)
{         if (type.equals("two")) {
if (twoWheelers > 0) {
twoWheelers--;
                System.out.println("Two wheeler left");
            }
        } else if (type.equals("four")) {
if (fourWheelers > 0) {
fourWheelers--;
                System.out.println("Four wheeler left");
            }
        }
    }

    public void display() {
        System.out.println("Two-wheelers: " + twoWheelers);
        System.out.println("Four-wheelers: " + fourWheelers);
System.out.println("Revenue: " + revenue);
    }

    public static void main(String[] args) {

        Parkinglot p = new Parkinglot(2, 2);

        p.park("two");
        p.park("two");
        p.park("two");

        p.park("four");
```

```
p.park("four");
p.park("four");

p.leave("two");
p.leave("four");
```

```
        p.display();
    }
}
```

Output :

```
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> javac Pract2_3.java
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> java Pract2_3.java
Two-wheeler parked
Two-wheeler parked
Two-wheeler section Full
Four-wheeler parked
Four-wheeler parked
Four-wheeler section Full
Two-wheeler left
Four-wheeler left
Two-wheelers: 1
Four-wheelers: 1
Revenue: 120
```

PART B:

Part B — Project Work (Hour 2): MiniBank

In Practical 1 you built the empty menu shell. Now you create the data the bank works on —
the Customer and Account classes. Using encapsulation, an account's balance can change
only through deposit() and withdraw(), never directly. These two classes are the foundation every later lab extends.

1. Create a class named Customer with private fields name, email and mobile (all String) and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as
"CUST101". Add a constructor and public getter methods.
2. Create a class named Account with private fields: a final String accountNumber, a String ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example "AC0001").
3. Add two constructors to Account: one taking ownerName and an opening balance; one taking only ownerName that calls the first using this(ownerName, 0).
4. Add a public method deposit(long amount) that adds amount to balance. Add a public
method withdraw(long amount) that subtracts amount and returns true when the balance is sufficient, otherwise returns false without changing the balance. Add getter methods.
Do NOT add a public setter for balance.
5. In main, create three Account objects inside an Account[] array, perform a few deposits and withdrawals, and print each balance.

sssCODE:

```
class Customer {

   private String name;     private
String email;     private String
mobile;     private final String
customerId;

   private static long customerCounter = 101;

   private static String generateCustomerId() {
return "CUST" + customerCounter++;
   }

   public Customer(String name, String email, String mobile)
{        this.name = name;         this.email = email;
this.mobile = mobile;         this.customerId =
generateCustomerId();
   }

   public String getName() {
return name;
```

```
    }

    public String getEmail() {
return email;
    }

    public    String    getMobile()    {
return mobile;
    }

    public String getCustomerId() {
return customerId;
    }
}

class Account {

    private final String
accountNumber;     private String
ownerName;     private long balance;
private boolean active;
```

```
    private static int counter = 1;
```

```
    private static String generateAccountNumber() {
return String.format("AC%04d", counter++);
    }

    public Account(String ownerName, long balance) {
this.accountNumber = generateAccountNumber();
this.ownerName = ownerName;         this.balance =
balance;         this.active = true;
    }

    public Account(String ownerName) {
this(ownerName, 0);
    }

    public void deposit(long amount)
{         if (amount > 0)
balance += amount;
    }

    public boolean withdraw(long amount) {
if (amount <= balance) {
```

25AIML006
CEUE203-Object Oriented Programming

```
            balance -= amount;
            return true;
        }
        return false;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getOwnerName() {
        return ownerName;
    }

    public long getBalance() {
        return balance;
    }

    public boolean isActive() {
        return active;
    }
}
```

```
public class MiniBank {

    public static void main(String[] args) {

        Customer c1 = new Customer("Daksh",
"daksh18@gmail.com", "9909900018");
        Customer c2 = new Customer("Kartavya",
"kartavya31@gmail.com", "9909900031");
        Customer c3 = new Customer("meet", "meet13@gmail.com",
"9909900013");

        System.out.println(c1.getCustomerId() + " " + c1.getName());
         System.out.println(c2.getCustomerId() + " " + c2.getName());
      System.out.println(c3.getCustomerId() + " " + c3.getName());

        Account[] accounts = new Account[3];

        accounts[0] = new Account("Daksh", 500000);
accounts[1] = new Account("Kartavya");
accounts[2] = new Account("meet", 350000);

        accounts[0].deposit(100000);
accounts[1].deposit(250000);
accounts[2].withdraw(150000);
```

```
        accounts[0].withdraw(1700000);


        for (Account acc : accounts) {
            System.out.println("Account No : " +
acc.getAccountNumber());
            System.out.println("Owner     : " + acc.getOwnerName());
            System.out.println("Balance   : " + acc.getBalance());
            System.out.println();
```

```
PS C:\Users\Daxha\OneDrive\Apps\Desktop\java> javac MiniBank.java
>> java MiniBank
CUST101 Daksh
CUST102 Kartavya
CUST103 meet
Account No : AC0001
Owner      : Daksh
Balance    : 600000

Account No : AC0002
Owner      : Kartavya
Balance    : 250000

Account No : AC0003
Owner      : meet
Balance    : 200000

PS C:\Users\Daxha\OneDrive\Apps\Desktop\java>
```

-